

BranchLedger System Architecture

Go backend and Laravel Blade web application

Version 1.0 • 4 October 2026

Proposed system specification and operating procedures

BranchLedger supports daily accounting at the main branch and sub-branches, including offline entry and online synchronisation. Owners and general managers receive a clear company view with traceable financial records. This document defines intended behaviour for development and pilot validation. It does not certify an implemented product or statutory compliance.

Contents

Architecture decision

Components and ownership

Deployment and data paths

Data model and database constraints

API contracts and state transitions

Synchronisation sequence and conflicts

Accounting transaction boundary

Implementation layout and delivery

Architecture diagrams and technical sources

Architecture decision

Use Go for the central API, accounting engine, synchronisation, reporting queries and background jobs. Use Laravel in PHP with Blade templates for the web presentation layer and session gateway. Blade is a template engine, not a standalone programming language. JavaScript provides offline browser behaviour and charts.

Choose a modular monolith for the Go backend initially. A single deployable backend simplifies transactional consistency and operational support. Split services only after measured workload or organisational needs justify it. PostgreSQL is the proposed authoritative database; compatible supported software versions must be pinned during implementation.

Components and ownership

Component	Responsibility	Data ownership
Browser PWA	Cached forms, chart interaction, IndexedDB and outbox	Local pending records and authorised snapshot
Laravel and Blade	HTML, sessions, CSRF and same-origin API gateway	Session metadata only

Component	Responsibility	Data ownership
Go API	Access control, validation, posting and sync	Authoritative business rules
PostgreSQL	Source records, journals, stock, audit and cursors	Authoritative company records
Go workers	Summaries, exports, alerts and attachment processing	Job results and projection state
Object storage	Receipts, attachments and exports	Protected objects linked to company IDs
Monitoring and backups	Health, metrics, restore and incident evidence	Operational records

Browser requests use the same HTTPS origin. Laravel routes /api requests through an authenticated gateway to Go, with verified identity context and request IDs. Go independently validates the delegated identity and scopes. Reject client-provided identity headers. Laravel never writes financial tables or reimplements posting rules.

Deployment and data paths

Online path: browser requests a Blade page from Laravel, then reads and submits business data through the same-origin gateway to Go. Go accesses PostgreSQL and object storage. Workers consume committed outbox jobs. The owner dashboard uses authorised server reports and shows the freshness of each branch.

Offline path: the service worker serves the previously installed application shell. JavaScript reads the local authorised snapshot and writes transaction plus outbox entry in IndexedDB. A foreground reconnect or manual Sync action sends queued operations. Background sync is an enhancement because browser support and execution are not guaranteed.

Host a TLS reverse proxy, Laravel process, Go API, workers and database in separated deployment units. Keep PostgreSQL and worker endpoints private. Use independent least-privilege credentials. Health checks distinguish process liveness from readiness to accept work. Keep secrets outside source code and logs.

Data model and database constraints

Entity	Essential fields or constraints
companies and branches	Company ID; branch belongs to exactly one company
users and memberships	Company membership, role, branch scopes and active state
devices	Company, branch, enrolment and authorisation lease
operations	Company plus operation ID unique; payload hash; durable result
transactions	Source reference, currency, status, revision and branch
journals and journal_lines	Company, source, account, debit or credit, reporting amount
accounts and periods	Company chart, account type and period lock
products recipes and stock_movements	Units, costs, recipe versions and linked source
transfers	Sender, receiver, in-transit state and confirmations

Entity	Essential fields or constraints
attachments	Company scope, checksum and upload state
audit_events and sync_changes	Actor, server time, record version and change cursor
outbox_jobs	Commit-linked event, status and retry information

Use company-scoped foreign keys to prevent cross-company references. Add unique keys for operation IDs and appropriate supplier-invoice identity. Similarity checks flag possible manual duplicates without treating every equal amount as a duplicate. Add indexes beginning with company and relevant branch/date fields; use keyset pagination for long record lists.

API contracts and state transitions

Method and route	Contract
POST /api/v1/sync/push	Batch operations with operation ID, hash and expected version
GET /api/v1/sync/pull?cursor=...	Authorised changes and next opaque cursor
GET /api/v1/dashboard	Branch/date/currency scope and freshness metadata
POST /api/v1/transactions	Guided command with idempotency key
POST /api/v1/transactions/{id}/reverse	Reason and linked reversal command
POST /api/v1/closes	Cash count, expected total and discrepancy
POST /api/v1/approvals/{id}/decision	Decision, reason and current record version
POST /api/v1/transfers/{id}/receive	Authorised receiver confirmation
POST /api/v1/exports	Creates background export job
GET /api/v1/exports/{id}	Job state and authorised download

Each push response includes operation ID, accepted or rejected state, server record ID, current version and a stable error code. HTTP 409 represents an incompatible revision or reused key with different content. Batch items can succeed independently; each accepted financial operation remains atomic. Rejected records are retained locally.

Business states: Draft, Awaiting approval, Posted, Reversed or Rejected. Sync states: Pending, Syncing, Synced or Failed. A synced draft is still a draft, and a local submitted record is not centrally posted until validation succeeds.

Synchronisation sequence and conflicts

1. Validate the local form and save transaction plus outbox atomically.
2. Establish an authorised session after reconnect.
3. Send pending operations with stable IDs.
4. Go checks permissions, duplicate key and expected revision.
5. Commit posting, audit, change event and acknowledgement.
6. Browser records acknowledgement and pulls server changes.
7. Apply change batch and cursor atomically.
8. Compare counts, totals and unresolved items.

A lost acknowledgement causes a retry with the same operation ID, which returns the original committed result. Never implement exactly-once transport claims; use at-least-once delivery with idempotent processing. Conflicting drafts require an authorised resolution. Posted entries remain immutable and use reversal or adjustment.

Inventory and cash closing use the single designated offline writer policy initially. A shared LAN branch service is a future option and requires secure local enrolment, HTTPS or trusted local origins, backup and coordinator-failure handling. Cloud reconciliation remains necessary.

Accounting transaction boundary

Go begins a database transaction, checks the posting period and source version, derives journal lines, validates balancing, applies stock valuation, persists source and ledger entries, writes audit and outbox rows, stores the operation result and commits. Rollback removes the complete attempted posting if validation or persistence fails.

External effects such as emails, exports and uploads do not occur inside the financial database commit. A worker processes the committed outbox and uses its own idempotency keys. Use reviewed isolation and row locking for stock and balance contention, with bounded retries for deadlocks.

Implementation layout and delivery

Proposed repository directories: backend/cmd/api, backend/cmd/worker, backend/internal/auth, accounting, sync, inventory, reporting and tenancy; web/resources/views, web/resources/js/offline and web/resources/js/charts; database/migrations; contracts/openapi; tests/fixtures; ops/deploy and docs.

CI runs formatting, static analysis, dependency checks, meaningful unit tests, database integration tests and access-isolation tests. Release database migrations using expand-and-contract changes. Version sync schemas and support an agreed older client window. Back up pending local records before client storage migrations; do not force reload during an unsaved task.

Initial release gates cover accounting review, failure-injection sync tests, tenant isolation, restore and browser migration. Performance targets and branch/device capacity remain proposed until pilot measurements establish an operating envelope.

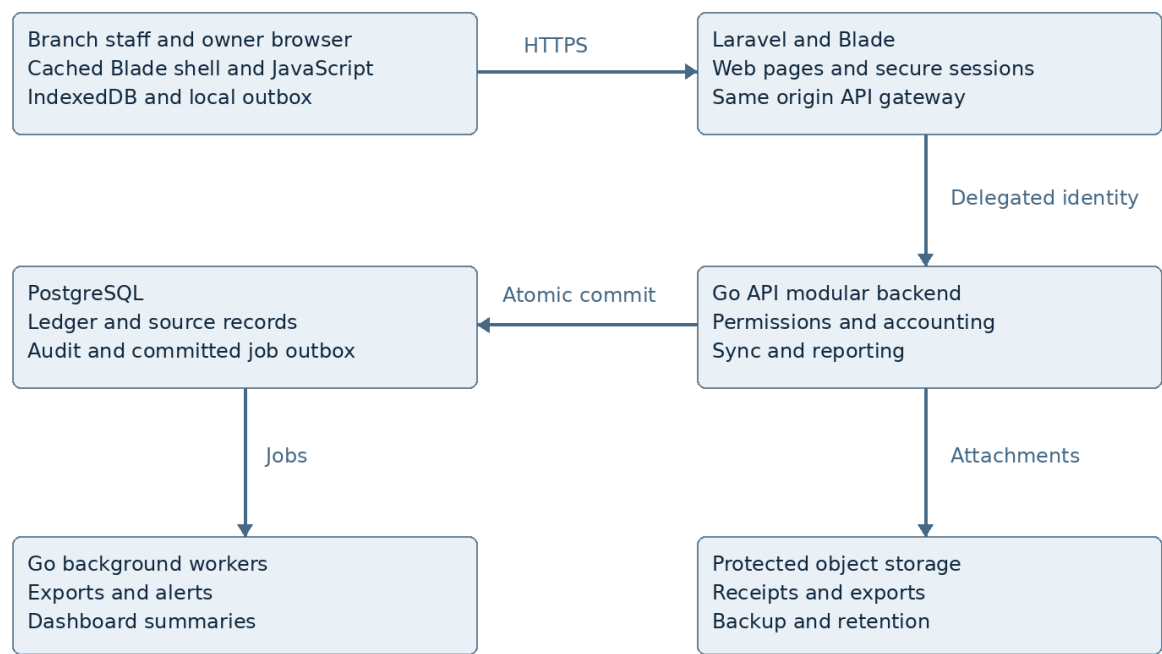
Architecture diagrams and technical sources

The component diagram shows the proposed deployment relationships. The preceding sections describe the synchronisation sequence and core data relationships. This is a proposed architecture rather than an already deployed system.

Official references checked 4 October 2026: Laravel Blade documentation at <https://laravel.com/docs/blade>; Go transactions at <https://go.dev/doc/database/execute-transactions>; MDN IndexedDB at https://developer.mozilla.org/en-US/docs/Web/API/IndexedDB_API/Using_IndexedDB; MDN service workers at https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API/Using_Service_Workers. These support the technology capabilities; BranchLedger design decisions remain proposed.

Component architecture diagram

BranchLedger component architecture



Offline: cached forms save locally. Online: Go validates and acknowledges.
Single offline writer per branch initially. Central database owns posted records.

The browser stores pending records locally. Laravel provides presentation and the secure gateway. Go exclusively owns financial rules and writes authoritative records. Workers process committed jobs.

BranchLedger launch safeguards update

Version 1.1 addendum | 4 October 2026. This addendum forms part of this PDF and updates the original requirements wherever they differ. Repeated checks have been merged. Items 1 to 39 cover technical launch controls, 40 to 49 cover presentation and customer information, and 50 adds the requested temporary account-lockout control.

All controls are required for implementation or verification, not reported as completed. No application source code or deployed service was supplied for this update. The PDFs therefore do not prove that BranchLedger is secure, free of defects or ready for launch. Retain the original accounting, offline and branch requirements.

Go and Laravel implementation ownership

Reverse proxy: terminate TLS, redirect HTTP to HTTPS, apply reviewed HSTS, limit request bodies and headers, disable directory listing, and deny private paths. Enable includeSubDomains or preload only after validating every affected host. HSTS is delivered on HTTPS responses.

Laravel: own login sessions and account recovery using maintained hashing and password-broker facilities. Rotate session IDs after login; revoke active sessions and delegated credentials after password changes. Put cookie-authenticated mutations behind CSRF middleware and keep the browser API same-origin. Secure cookies must use HttpOnly and an appropriate SameSite setting.

Go: enforce identity, tenant and branch scope independently on every command. Validate types and accounting rules, bound body reads and batch sizes, propagate context deadlines, parameterise SQL and use atomic posting with unique operation IDs. Check quota and rate limits without trusting client headers or client prices.

Database: separate migration credentials from runtime credentials, scope caches and queries to company and permissions, add reviewed indexes, paginate large lists and store reset tokens as hashes. Never make production database ports public. Do not cache sensitive shared responses without explicit user scope and invalidation.

Billing and integration boundaries

Verify provider webhook signatures against the exact raw request bytes before parsing. Check the provider event identity, replay policy, amount, currency, merchant and target company. Persist unique provider event and payment identifiers. A browser success redirect alone never activates a subscription or settles a financial payment. Reconcile uncertain provider outcomes before retrying.

The server selects subscription prices and plan entitlements. For business sales, use server-authorized price lists and discount rules; intentional overrides require permission and audit. Offline sales preserve the cached price-list version and may require conflict review. Record history rather than silently replacing accepted prices.

Upload and stored data safety

Allowlist intended receipt and import formats and check extension, MIME and actual content. Use generated object names, tenant-scoped access, size and decompression limits, malware checks where applicable, quarantine and non-executable storage outside the web root. Authorise each download. CSV imports require schema validation and spreadsheet exports require

formula-injection protection.

Validate and normalise input deliberately. Escape for the output context, use parameterised database queries and sanitise any deliberately supported rich HTML. Blanket removal of characters before storage can damage names, references and financial evidence and does not replace these controls.

Abuse resistance and optional AI

Throttle login and password reset by account and network signals with generic responses. Use progressive delays or temporary lockouts, monitored recovery and stronger authentication. Avoid permanent attacker-triggered denial of access. Password-reset tokens expire and become unusable after successful reset.

If AI is enabled, retrieved receipts and messages are untrusted inputs. Restrict company scope, allowlist tools, cap usage and require human approval for consequential actions. AI must not directly post journals, approve requests or alter permissions. Test prompt-injection attempts; no filter guarantees complete prevention. Keep AI disabled for the initial accounting release.

Operations and failure tests

Scan Go modules, Composer packages and browser dependencies in CI. Monitor uptime, errors, queue age and spending. Redact secrets and unnecessary personal information from logs. Log access denials, role changes and exports. Set provider caps plus application budget enforcement; alerts alone are not hard spending limits.

Test 429 throttling, 413 body rejection, timeouts, duplicate callbacks, invalid CSRF, expired reset tokens, cache isolation and temporary lock recovery. Server permission changes are enforced on reconnect. A revoked offline device cannot be instantly updated; preserve its records for authorised review without bypassing current permissions.

Reference guidance and scope

Official guidance reviewed 4 October 2026. These references support control design; they do not certify BranchLedger. Authentication and safe recovery:

https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html and

https://cheatsheetseries.owasp.org/cheatsheets/Forgot_Password_Cheat_Sheet.html. Upload

handling: https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html.

Optional AI: https://cheatsheetseries.owasp.org/cheatsheets/LLM_Prompt_Injection_Prevention_Cheat_Sheet.html.